Foundations of Machine Learning - Assignment 1 Report

Name: _____ Roll number: _____

Code files: **q1.py** (Question 1) and **q2.py** (Question 2). Keep Dataset1.csv and Dataset2.csv in the same folder and run "python q1.py" and "python q2.py". All the maths is written by hand with loops. numpy is only used for eigenvalues/eigenvectors (np.linalg.eigh) and matplotlib only for plots.

Note: I used an AI assistant (Claude) to help me with the code and this report, as allowed by the instructor.

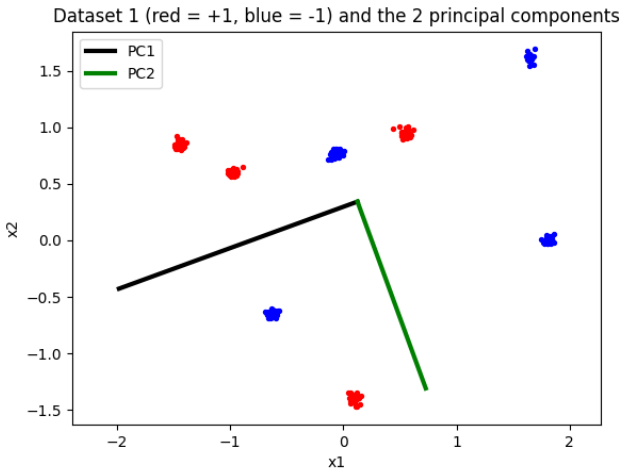
Question 1

(a) PCA

Steps: find the mean of the data, subtract it from every point, make the 2x2 covariance matrix, and find its eigenvalues and eigenvectors. The eigenvectors are the principal components (PCs) and each eigenvalue tells how much spread (variance) is along that PC.

	Direction	Eigenvalue	Variance explained
PC1	(-0.939, -0.343)	1.258	61.95 %
PC2	(0.343, -0.939)	0.772	38.05 %

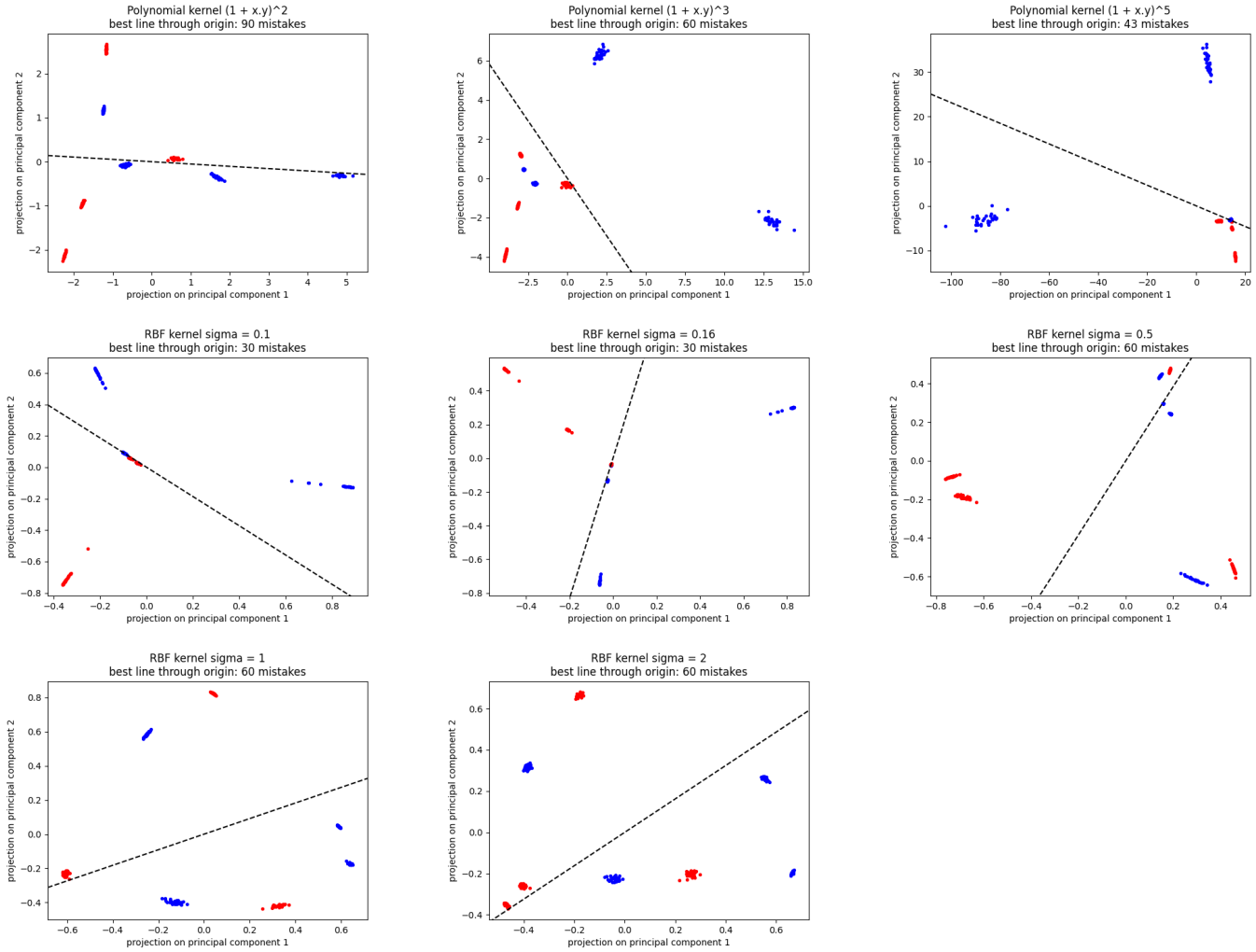
So PC1 explains about 62% and PC2 about 38% of the variance. The two numbers are not very different, which means the data is spread out in both directions and no single straight direction captures it well. Also, the data is 8 small groups with red (+1) and blue (-1) mixed around, so a straight line direction alone can not separate the classes.



(b) Kernel PCA

Steps: make the n x n kernel matrix K (K[i][j] = k(x_i, x_j)), centre it (so that the data is centred in the feature space), find the top 2 eigenvalues and eigenvectors, and the projection of point i on component j is sqrt(eigenvalue$_j$) x (eigenvector$_j$)[i].

Kernels tried: polynomial $(1 + x.y)^d$ for d = 2, 3, 5, and RBF $\exp(-||x - y||^2 / 2\ \text{sigma}^2)$ for sigma = 0.1, 0.16, 0.5, 1, 2. The plots are below (red = +1, blue = -1). The dashed line is the best line through the origin I could find (used in part c).



What I noticed: every small group of 30 points stays together as a tight dot in all plots, but the red and blue dots stay mixed. Polynomial kernels stretch the far groups a lot (bigger degree = more stretching) and the close groups get squashed near the centre. For RBF with a very small sigma (0.1, 0.16), every group becomes "its own island", all 8 groups look alike to the kernel, so the top eigenvalues are almost equal (about 27 each). The top 2 components then just pick 2 or 3 groups and the rest pile up in the middle. For larger sigma (1, 2) the picture looks like a smooth bent version of the original data.

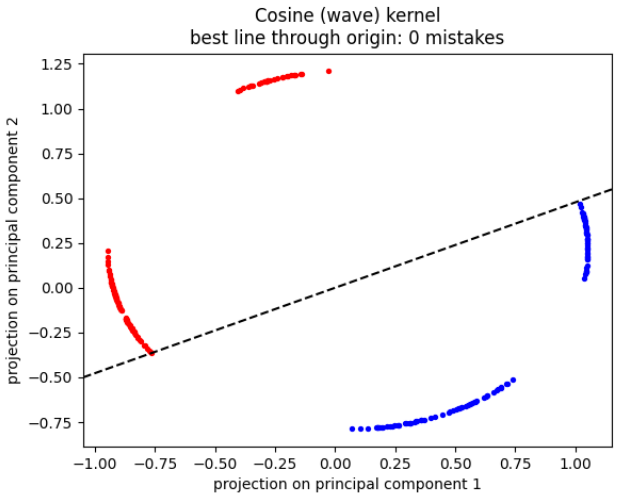
(c) A kernel that makes the data linearly separable

To check separability, for each plot I tried every direction w (0 to 360 degrees in steps of 0.5 degree) and counted how many points break $\text{sign}(w^T z) = y$. 0 mistakes means linearly separable. None of the kernels in part (b) reached 0. The best was RBF with small sigma (30 mistakes, which is just one group on the wrong side).

Yes, such a kernel exists. If I go along the x1 direction (tilted a little), the groups come in the order red, red, blue, blue, red, red, blue, blue. It goes up and down like a wave. So I used a cosine (wave) kernel:

k(x, y) = cos(3.3 (x1 - y1) - 0.6 (x2 - y2))

This is a valid kernel. If t = 3.3 x1 - 0.6 x2, then $k(x, y) = \cos(t_x)\cos(t_y) + \sin(t_x)\sin(t_y)$, which is just the dot product of the features (cos t, sin t). So every point is sent to a spot on a circle. The numbers 3.3 and 0.6 set how fast the wave goes, and I chose them by trying values and picking the ones that give the biggest gap (values from about 3.2 to 3.4 also work). With this kernel the red groups land on one half of the circle and blue on the other half. After kernel PCA, the top 2 components give **0 mistakes** with a line through the origin, so the data becomes linearly separable.


Cosine (wave) kernel
best line through origin: 0 mistakes
projection on principal component 2
projection on principal component 1
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

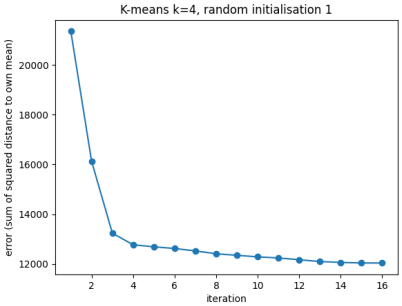
Question 2

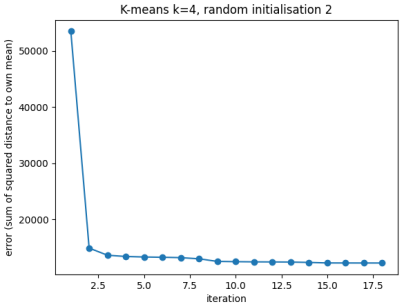
The data is 1000 points lying on 4 circles (rings) around the origin, with radius about 1, 3, 6 and 9 (250 points on each).

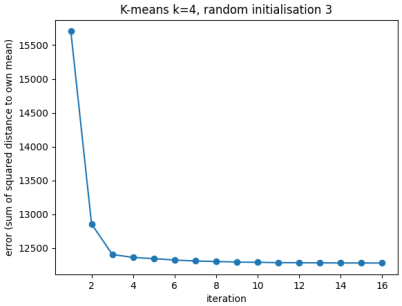
(a) K-means with k = 4, five random starts

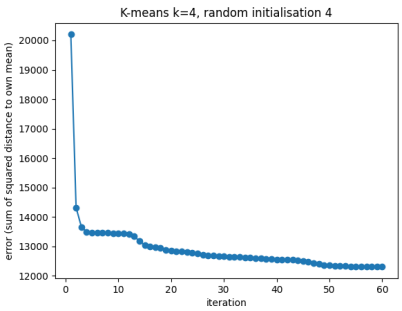
Algorithm (Lloyd's): pick 4 random data points as starting means. Then repeat: (1) give every point to the closest mean, (2) move each mean to the average of its points. Stop when no point changes cluster. Error = sum of squared distance of every point from its own mean.

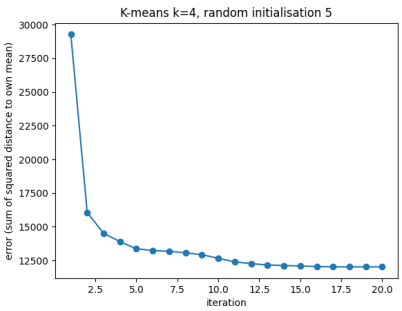
Start	1	2	3	4	5
Iterations	16	18	16	60	20
Final error	12033	12211	12278	12299	12030

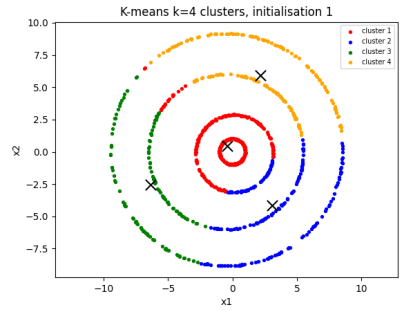


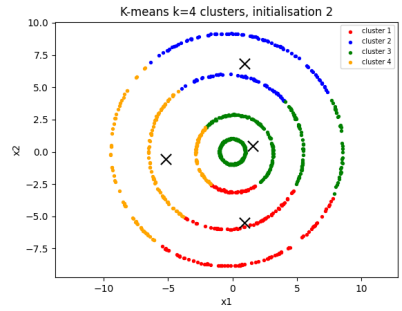


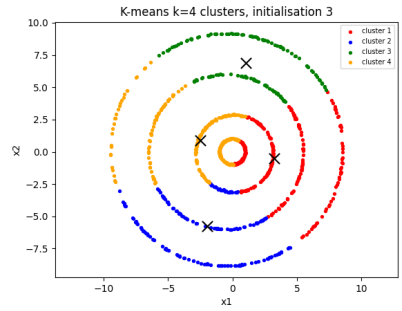


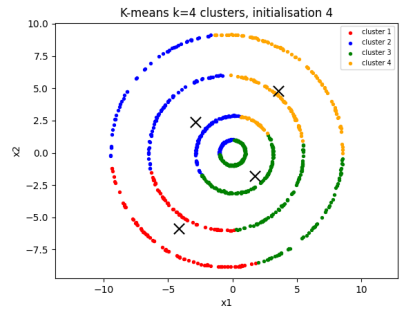


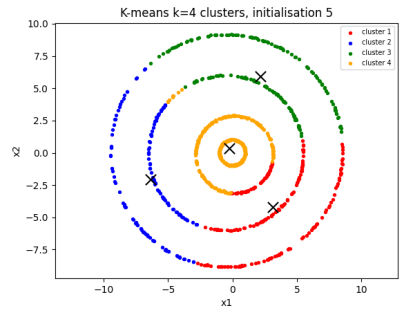








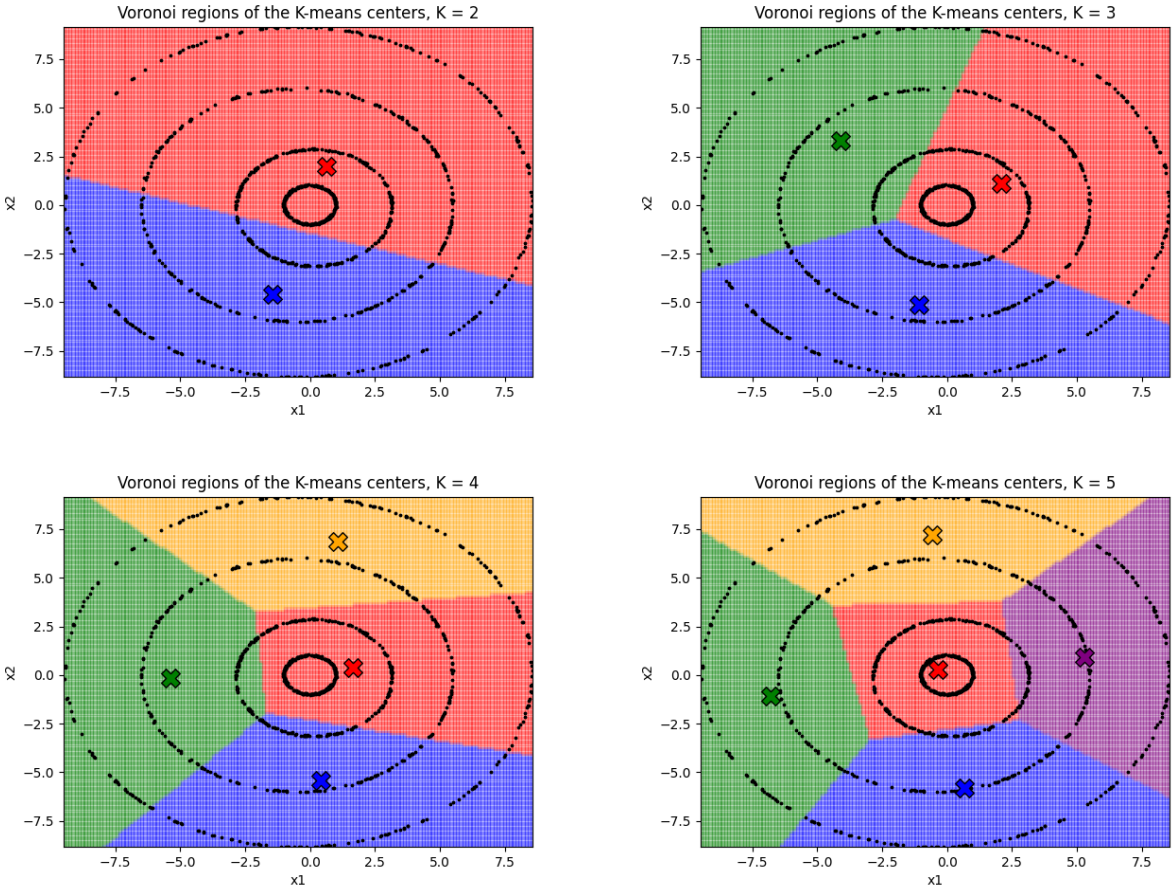




What I noticed: the error goes down in every iteration and never goes up. It drops a lot in the first 2-3 steps and then slowly settles. Different starts end at different final errors and different clusters, so K-means gets stuck at a local best answer, not always the overall best. In all 5 cases the clusters do not follow the rings. K-means only uses the distance to a mean, so it cuts the picture into pie-like pieces with straight borders and each cluster mixes parts of several rings.

(b) Voronoi regions for K = 2, 3, 4, 5

I fixed one random start (seed 100), ran K-means, and then coloured every point of a fine grid (from the min to the max value of the data in each coordinate) by its nearest center. Each coloured area is the Voronoi region of that center (the crosses).



The borders between regions are always straight lines (each one is halfway between two centers). Because of this, K-means can never give ring shaped clusters for this data, whatever K is. The rings are always cut across.

(c) Spectral clustering with k = 4

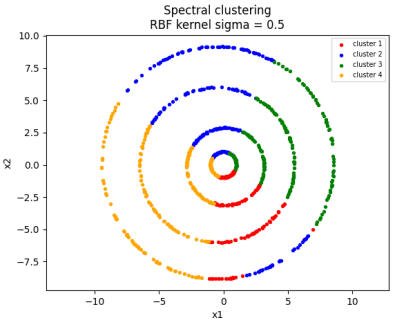
Steps: make the kernel matrix K, take its top 4 eigenvectors v1..v4, write each point i as the row (v1[i], v2[i], v3[i], v4[i]), make each row have length 1, and run K-means on these rows (best of 5 starts). The cluster of the row is the cluster of the point.

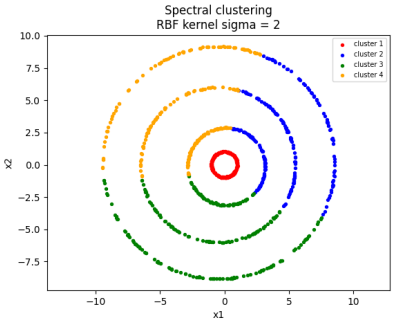
First I tried the normal RBF kernel (sigma = 0.5 and 2). It did not work well: with sigma = 0.5 the rings get broken into arcs, and with sigma = 2 only the small rings come out right. The reason is that on the big rings the points are far apart, and points on nearby rings can be closer to each other than to points on the other side of their own ring.

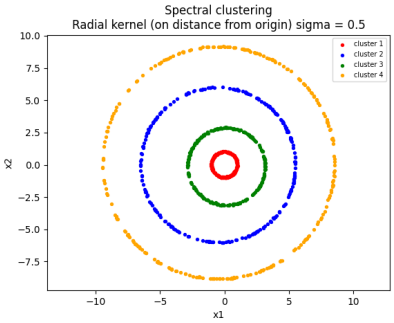
So I chose a kernel that looks only at the distance from the origin, r = sqrt($x1^2 + x2^2$):

$k(x, y) = \exp(-(r_x - r_y)^2 / (2 \times 0.5^2))$

Points on the same ring have almost the same r, so the kernel value is close to 1. Points on different rings have r differing by 2 or more, so the value is almost 0. This kernel gives **exactly the 4 rings** as the 4 clusters (250 points each). The output confirms this is the right kernel for ring shaped data.

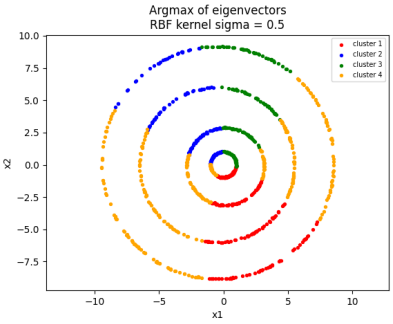


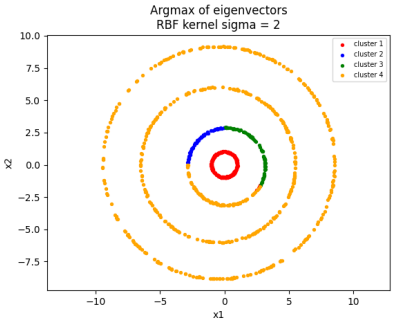


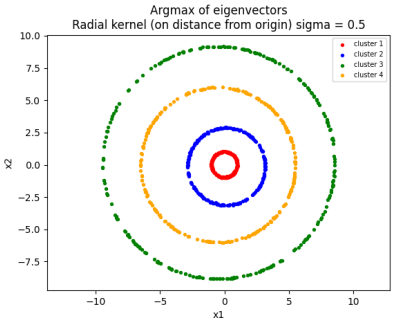


(d) Assigning by the largest eigenvector entry (argmax)

Here point i goes to cluster l if $v_l[i]$ is the biggest among v1[i]..v4[i]. Since an eigenvector can come with either sign, I flipped each one so its biggest entry is positive.







With the radial kernel it works perfectly, same as part (c): 250 points in each cluster, one ring each. With the normal RBF kernel it works badly (for sigma = 2 one cluster gets 592 points and the outer rings are mixed).

Why: with the radial kernel the kernel matrix is almost made of 4 separate blocks (one block per ring, near zero outside). Then each top eigenvector is almost "a constant on one ring and about 0 on the others", so the biggest entry directly tells the ring. With the normal RBF kernel the groups are not cleanly separated in K, so the eigenvectors spread over many rings and have both positive and negative parts, and the biggest entry does not match a ring. So this simple argmax trick only works when the kernel already separates the clusters very clearly. The method in part (c) (normalise rows and run K-means) is safer in general.